

Déploiement Automatisé d'une Infrastructure Cloud avec Terraform

Rapport de Projet – Cloud Computing - Infrastructure as Code

Haidar ESBER

EFREI Paris B3 Développeur fullstack

Date : Mars 2026

Provider Cloud : Microsoft Azure

Technologies : Terraform · Flask · Python · Azure VM · Blob Storage · PostgreSQL

Accueil

rg-flask-app Groupe de ressources

Comment gérer ces modifications plus efficacement avec les outils de déploiement ? M'aider à générer Laforme pour cette configuration de groupe de ressources. Générez du code Bicep pour dupliquer ce groupe de ressources.

Rechercher

+ Créer Gérer l'affichage Supprimer le groupe de ressources Actualiser Exporter au format CSV Ouvrez une requête Attribuer des balises Déplacer Supprimer

Aucun regroupement

Vue d'ensemble

Journal d'activité

Contrôle d'accès (IAM)

Étiquettes

Visualiseur de ressources

Événements

Paramètres

Gestion des coûts

Supervision

Automatisation

Aide

^ Bases

Abonnement (déplacer) : Azure for Students

ID d'abonnement : 712887c6-9bf0-4d2e-adbc-fbb46c3c36db

Déploiements : Aucun déploiement

Emplacement : France Central

Étiquettes (modifier) : Environment : dev ManagedBy: Terraform Afficher plus (1)

Ressources Recommandations

Filtrer un champ... Type est égal à tout Emplacement est égal à tout Ajouter un filtre

Nom	Type	Emplacement
☐ flaskapp-nic	Interface réseau	France Central
☐ flaskapp-nsg	Groupe de sécurité réseau	France Central
☐ flaskapp-postgres	Serveur flexible Azure Database pour PostgreSQL	France Central
☐ flaskapp-public-ip	Adresse IP publique	France Central
☐ flaskapp-vm	Machine virtuelle	France Central
☐ flaskapp-vm_OsDisk_1_0843175b500d4b4aa5af3920e0a89a36	Disque	France Central
☐ flaskapp-vnet	Réseau virtuel	France Central
☐ flaskapp.postgres.database.azure.com	Zone DNS privé	Global
☐ flaskappstorage61atxp	Compte de stockage	France Central

Table des matières

1. Introduction et Objectifs	4
2. Architecture de l'Infrastructure.....	4
2.1 Schéma global.....	4
2.2 Ressources Terraform (16 au total).....	5
3. Étape 1 — Préparation de l'Environnement Terraform	6
3.1 Installation des outils.....	6
3.2 Structure des fichiers Terraform	6
3.3 Configuration du provider (provider.tf)	7
3.4 Initialisation — terraform init.....	7
4. Étape 2 — Déploiement de l'Infrastructure	8
4.1 Machine Virtuelle	8
4.2 Azure Blob Storage	9
4.3 Base de Données PostgreSQL.....	10
4.4 Sécurité Réseau (NSG)	11
4.5 Résultat terraform apply	11
5. Étape 3 — Backend Flask et Connexion au Stockage	12
5.1 Endpoints API	12
5.2 Modèle de données PostgreSQL	12
5.3 Connexion Azure Blob Storage	12
5.4 Variables d'environnement sur la VM	12
6. Étape 4 — Automatisation du Déploiement	13
6.1 Variables dynamiques (variables.tf)	13
6.2 Provisioning automatique — Cloud-init	13
6.3 Service systemd	13
6.4 Outputs Terraform	14
7. Étape 5 — Tests et Validation	15
7.1 Health Check.....	15
7.2 CREATE — Upload d'un fichier	15
7.3 READ — Liste des fichiers et lecture de fichier par ID	15
7.4 UPDATE — Modification de la description.....	15
7.5 DELETE — Suppression.....	16

7.6 Destruction de l'infrastructure	16
8. Problèmes Rencontrés et Solutions	17
Problème 1 — Service Principal non autorisé	17
Problème 2 — Régions non autorisées (Azure Policy)	17
Problème 3 — Tailles de VM indisponibles (Capacity Restrictions)	17
Problème 4 — Conflit PostgreSQL Public/Private Network	17
Problème 5 — Zone PostgreSQL non gérée par Terraform	18
Problème 6 — cloud-init : fichiers non créés (owner inexistant).....	18
Problème 7 — Table PostgreSQL inexistante au démarrage Unicorn.....	18
9. Conclusion	19
9.1 Bilan technique.....	19
9.2 Points clés appris.....	19
9.3 Suppression de l'infrastructure	19

1. Introduction et Objectifs

Ce projet a pour objectif de déployer automatiquement une infrastructure cloud complète sur Microsoft Azure en utilisant Terraform comme outil d'Infrastructure as Code (IaC).

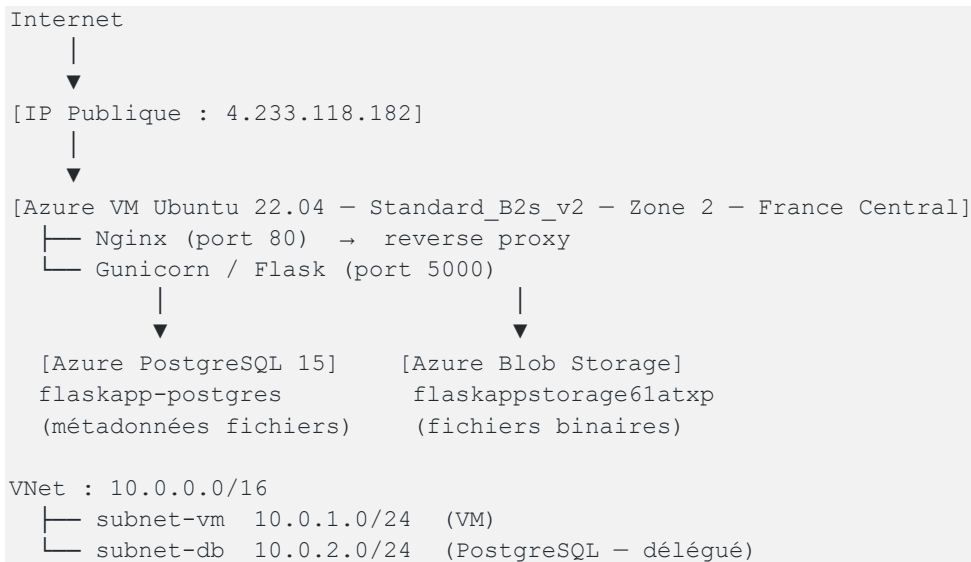
L'infrastructure déployée comprend :

- Une machine virtuelle Ubuntu 22.04 hébergeant une application web Flask
- Un Azure Blob Storage pour stocker les fichiers statiques (images, logs, etc.)
- Un serveur PostgreSQL managé pour la persistance des métadonnées
- Une API REST CRUD complète pour interagir avec le stockage et la base de données

L'ensemble du processus est automatisé : de la création des ressources cloud jusqu'au démarrage automatique de l'application en tant que service système.

2. Architecture de l'Infrastructure

2.1 Schéma global



2.2 Ressources Terraform (16 au total)

```
C:\Users\haida\computing\terraform-flask-azure\terraform>az resource list --resource-group rg-flask-app --query "[].{Nom:name, Type:type}" -o table
Nom                                         Type
-----
Flaskapp.postgres.database.azure.com      Microsoft.Network/privateDnsZones
Flaskapp-public-ip                       Microsoft.Network/publicIPAddresses
Flaskapp-nsg                             Microsoft.Network/networkSecurityGroups
Flaskapp-vnet                             Microsoft.Network/virtualNetworks
Flaskapp.postgres.database.azure.com/Flaskapp-dns-link Microsoft.Network/privateDnsZones/virtualNetworkLinks
Flaskapp-nic                             Microsoft.Network/networkInterfaces
Flaskappstorage61atxp                    Microsoft.Storage/storageAccounts
Flaskapp-postgres                        Microsoft.DBforPostgreSQL/FlexibleServers
Flaskapp-vm                              Microsoft.Compute/virtualMachines
Flaskapp-vm_OsDisk_1_0843175b500d4b4aa5af3920e0a89a36 Microsoft.Compute/disks
```

Ressource	Nom Azure	Description
azurerm_resource_group	rg-flask-app	Groupe de ressources
azurerm_virtual_network	flaskapp-vnet	VNet 10.0.0.0/16
azurerm_subnet (VM)	flaskapp-subnet-vm	Sous-réseau VM 10.0.1.0/24
azurerm_subnet (DB)	flaskapp-subnet-db	Sous-réseau PostgreSQL 10.0.2.0/24
azurerm_public_ip	flaskapp-public-ip	IP publique statique Standard
azurerm_network_security_group	flaskapp-nsg	Règles SSH/HTTP/Flask
azurerm_network_interface	flaskapp-nic	Interface réseau VM
azurerm_linux_virtual_machine	flaskapp-vm	Ubuntu 22.04, B2s_v2, Zone 2
azurerm_storage_account	flaskappstorage61atxp	Standard LRS
azurerm_storage_container	static-files	Container Blob privé
azurerm_private_dns_zone	flaskapp.postgres...	Zone DNS privée PostgreSQL
azurerm_private_dns_zone_virtual_network_link	flaskapp-dns-link	Lien DNS ↔ VNet
azurerm_postgresql_flexible_server	flaskapp-postgres	PostgreSQL 15, B_Standard_B1ms
azurerm_postgresql_flexible_server_database	flaskdb	Base UTF8
random_string	—	Suffixe unique pour le storage
azurerm_nic_nsg_association	—	Liaison NIC ↔ NSG

☐ Nom ↑	Type	Emplacement
☐  flaskapp-nic	... Interface réseau	France Central
☐  flaskapp-nsg	... Groupe de sécurité réseau	France Central
☐  flaskapp-postgres	... Serveur flexible Azure Database pou...	France Central
☐  flaskapp-public-ip	... Adresse IP publique	France Central
☐  flaskapp-vm	... Machine virtuelle	France Central
☐  flaskapp-vm_OsDisk_1_0843175b500d4b4aa5af3920e0a89a36	... Disque	France Central
☐  flaskapp-vnet	... Réseau virtuel	France Central
☐  flaskapp.postgres.database.azure.com	... Zone DNS privé	Global
☐  flaskappstorage61atxp	... Compte de stockage	France Central

3. Étape 1 — Préparation de l'Environnement Terraform

3.1 Installation des outils

Terraform et Azure CLI ont été installés via Chocolatey (gestionnaire de paquets Windows) :

```
choco install terraform azure-cli -y

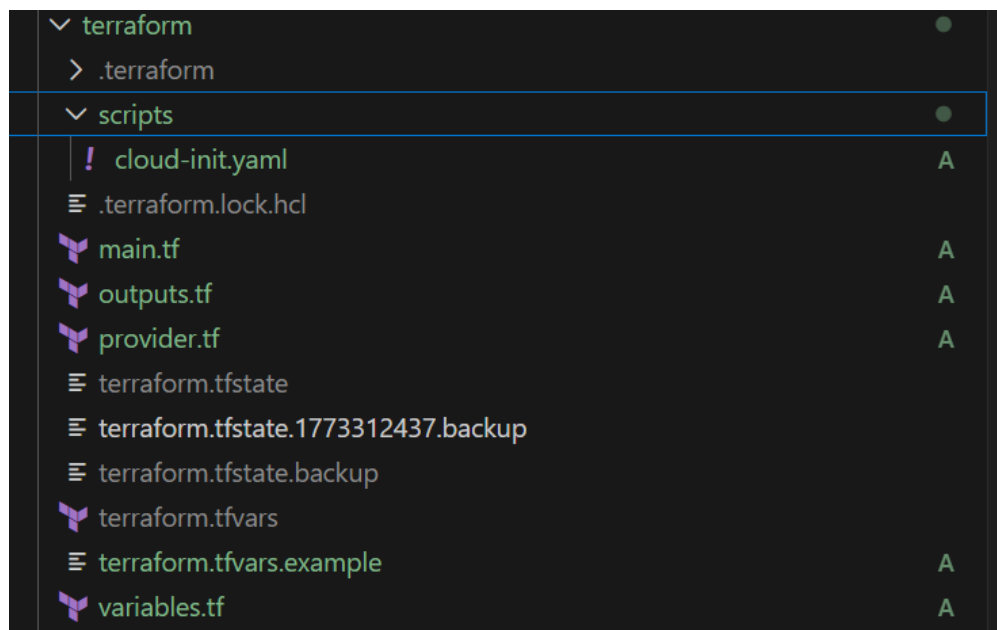
C:\Users\haida\ccomputing\terraform-flask-azure\terraform>terraform version
Terraform v1.14.6
on windows amd64

C:\Users\haida\ccomputing\terraform-flask-azure\terraform>az version
{
  "azure-cli": "2.84.0",
  "azure-cli-core": "2.84.0",
  "azure-cli-telemetry": "1.1.0",
  "extensions": {}
}
```

Versions installées : Terraform 1.14.6 · Azure CLI 2.x · Provider azurearm 3.117.1

3.2 Structure des fichiers Terraform

```
terraform/
├── provider.tf          # Configuration du provider Azure
├── main.tf              # Toutes les ressources Azure
├── variables.tf         # Déclaration des variables
├── outputs.tf           # Sorties (IP, URLs, FQDN)
├── terraform.tfvars     # Valeurs sensibles (non versionné)
├── terraform.tfvars.example
├── scripts/
│   └── cloud-init.yaml  # Script de provisioning VM
```



3.3 Configuration du provider (provider.tf)

```
provider.tf A X
terraform-flask-azure > terraform > provider.tf
1 terraform {
2   required_version = ">= 1.3.0"
3
4   required_providers {
5     azurerm = {
6       source = "hashicorp/azurerm"
7       version = "~> 3.90"
8     }
9   }
10 }
11
12 provider "azurerm" {
13   features {}
14
15   subscription_id = var.subscription_id
16   # Authentification via az login (pas de Service Principal requis)
17 }
18
```

3.4 Initialisation — terraform init

```
C:\Users\haida\ccomputing\terraform-flask-azure\terraform>terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.117.1
- Using previously-installed hashicorp/random v3.8.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

4. Étape 2 — Déploiement de l'Infrastructure

4.1 Machine Virtuelle

```
C:\Users\haida\ccomputing\terraform-flask-azure\terraform>az vm show -g rg-flask-app -n flaskapp-vm --query "{Nom:name, Taille:hardwareProfile.vmSize, OS:storageProfile.imageReference.offer, IP:publicIps}" -o table
Nom          Taille          OS
-----
flaskapp-vm   Standard_B2s_v2 0001-com-ubuntu-server-jammy

resource "azurerm_linux_virtual_machine" "main" {
  name           = "flaskapp-vm"
  location       = "France Central"
  size           = "Standard_B2s_v2"
  zone           = "2"
  admin_username = var.admin_username
  admin_password = var.admin_password

  source_image_reference {
    publisher = "Canonical"
    offer      = "0001-com-ubuntu-server-jammy"
    sku        = "22_04-lts-gen2"
    version    = "latest"
  }

  custom_data = base64encode(templatefile("scripts/cloud-init.yaml", { ... }))
}
```

IP publique obtenue après déploiement : 4.233.118.182

Propriétés

Supervision

Fonctionnalités (7)

Recommandations

Tutoriels

Machine virtuelle

Nom de l'ordinateur

flaskapp-vm

Système d'exploitation

Linux (ubuntu 22.04)

Génération de machine virtuelle

V2

Architecture de machine virtuelle

x64

État de l'agent

Ready

Version de l'agent

2.15.1.3

Mise en veille prolongée

Désactivé

Groupe hôte

-

Hôte

-

Groupe de placement de proximité

-

État de colocation

N/A

Groupe de réservations de capacité

-

Type de contrôleur de disque

SCSI

Mise en réseau

Adresse IP publique

4.233.118.182 (Interface réseau flaskapp-nic)

1 adresses IP publiques associées

Adresse IP publique (IPv6)

-

Adresse IP privée

10.0.1.4

Adresse IP privée (IPv6)

-

Réseau/sous-réseau virtuel

flaskapp-vnet/flaskapp-subnet-vm

Nom DNS

Configurer

Taille

Taille

Standard B2s v2

Processeurs virtuels

2

RAM

8 Gio

Détails d'image source

Éditeur d'image source

Canonical

Offre d'image source

0001-com-ubuntu-server-jammy

Plan d'image source

22_04-lts-gen2

Spot Azure

Connexion en SSH après la création pour exécuter les commandes bash :

```
PS C:\Users\haida> ssh azureuser@4.233.118.182
The authenticity of host '4.233.118.182 (4.233.118.182)' can't be established.
ED25519 key fingerprint is SHA256:LHEp/3YSK+XWzNjcv0SHGDgL3hGd8+7g95Azu02r7W4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '4.233.118.182' (ED25519) to the list of known hosts.
azureuser@4.233.118.182's password:
Permission denied, please try again.
azureuser@4.233.118.182's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-azure x86_64)

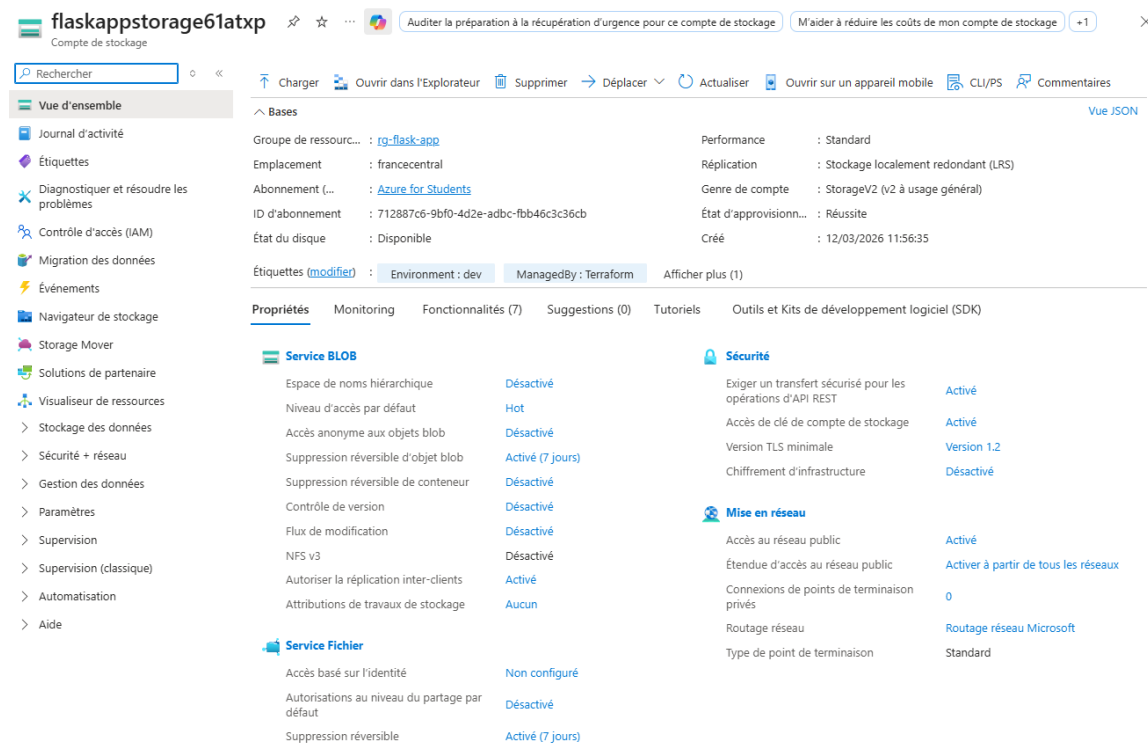
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

4.2 Azure Blob Storage

```
resource "azurerm_storage_account" "main" {
  name                        = "flaskappstorage61atxp"
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  allow_nested_items_to_be_public = false
}

resource "azurerm_storage_container" "static" {
  name                  = "static-files"
  container_access_type = "private"
}
```

Endpoint Blob : <https://flaskappstorage61atxp.blob.core.windows.net/>



The screenshot shows the Azure portal interface for the storage account 'flaskappstorage61atxp'. The left sidebar contains navigation options like 'Vue d'ensemble', 'Journal d'activité', 'Étiquettes', 'Diagnostic et résoudre les problèmes', 'Contrôle d'accès (IAM)', 'Migration des données', 'Événements', 'Navigateur de stockage', 'Storage Mover', 'Solutions de partenaire', 'Visualiseur de ressources', 'Stockage des données', 'Sécurité + réseau', 'Gestion des données', 'Paramètres', 'Supervision', 'Supervision (classique)', 'Automatisation', and 'Aide'. The main content area displays account details and configuration options.

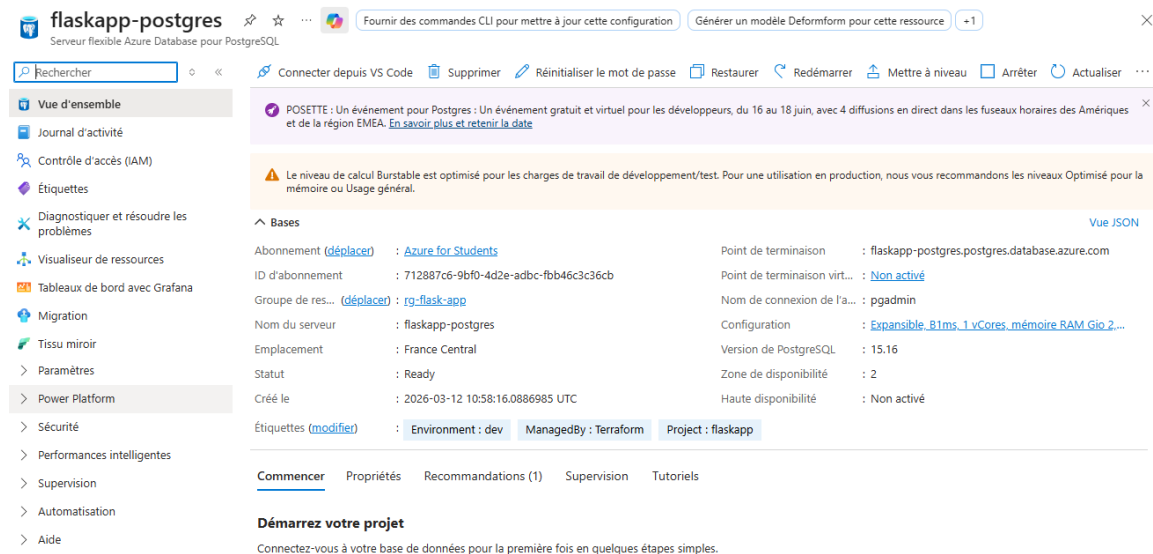
Propriétés	Monitoring	Fonctionnalités (7)	Suggestions (0)	Tutoriels	Outils et Kits de développement logiciel (SDK)
Service BLOB					
Espace de noms hiérarchique	Désactivé				
Niveau d'accès par défaut	Hot				
Accès anonyme aux objets blob	Désactivé				
Suppression réversible d'objet blob	Activé (7 jours)				
Suppression réversible de conteneur	Désactivé				
Contrôle de version	Désactivé				
Flux de modification	Désactivé				
NFS v3	Désactivé				
Autoriser la réplication inter-clients	Activé				
Attributions de travaux de stockage	Aucun				
Service Fichier					
Accès basé sur l'identité	Non configuré				
Autorisations au niveau du partage par défaut	Désactivé				
Suppression réversible	Activé (7 jours)				
Sécurité					
Exiger un transfert sécurisé pour les opérations d'API REST	Activé				
Accès de clé de compte de stockage	Activé				
Version TLS minimale	Version 1.2				
Chiffrement d'infrastructure	Désactivé				
Mise en réseau					
Accès au réseau public	Activé				
Étendue d'accès au réseau public	Activer à partir de tous les réseaux				
Connexions de points de terminaison privés	0				
Routeage réseau	Routeage réseau Microsoft				
Type de point de terminaison	Standard				

4.3 Base de Données PostgreSQL

```
resource "azurerm_postgresql_flexible_server" "main" {
  name           = "flaskapp-postgres"
  version        = "15"
  sku_name       = "B_Standard_B1ms"
  delegated_subnet_id = azurerm_subnet.db.id
  public_network_access_enabled = false
  storage_mb     = 32768

  lifecycle {
    ignore_changes = [zone]
  }
}
```

FQDN : flaskapp-postgres.postgres.database.azure.com



The screenshot shows the Azure portal interface for a PostgreSQL flexible server. The left sidebar contains navigation options like 'Vue d'ensemble', 'Journal d'activité', 'Contrôle d'accès (IAM)', 'Étiquettes', 'Diagnosticuer et résoudre les problèmes', 'Visualiseur de ressources', 'Tableaux de bord avec Grafana', 'Migration', 'Tissu miroir', 'Paramètres', 'Power Platform', 'Sécurité', 'Performances intelligentes', 'Supervision', 'Automatisation', and 'Aide'. The main content area displays the server's configuration details, including subscription, resource group, name, location, status, and creation time. It also shows a 'Bases' section with a table of server properties and a 'Commencer' section with a 'Démarez votre projet' link.

flaskapp-postgres    Fournir des commandes CLI pour mettre à jour cette configuration Générer un modèle Deformform pour cette ressource +1

Seur flexible Azure Database pour PostgreSQL

Rechercher

Connecter depuis VS Code Supprimer Réinitialiser le mot de passe Restaurer Redémarrer Mettre à niveau Arrêter Actualiser

Vue d'ensemble

Journal d'activité

Contrôle d'accès (IAM)

Étiquettes

Diagnosticuer et résoudre les problèmes

Visualiseur de ressources

Tableaux de bord avec Grafana

Migration

Tissu miroir

Paramètres

Power Platform

Sécurité

Performances intelligentes

Supervision

Automatisation

Aide

POSETTE : Un événement pour Postgres : Un événement gratuit et virtuel pour les développeurs, du 16 au 18 juin, avec 4 diffusions en direct dans les fuseaux horaires des Amériques et de la région EMEA. [En savoir plus et retenir la date](#)

Le niveau de calcul Burstable est optimisé pour les charges de travail de développement/test. Pour une utilisation en production, nous vous recommandons les niveaux Optimisé pour la mémoire ou Usage général.

Bases [Vue JSON](#)

Abonnement (déplacer)	: Azure for Students	Point de terminaison	: flaskapp-postgres.postgres.database.azure.com
ID d'abonnement	: 712887c6-9bf0-4d2e-adbc-fbb46c3c36cb	Point de terminaison virt...	: Non activé
Groupe de res... (déplacer)	: rg-flask-app	Nom de connexion de l'a...	: pgadmin
Nom du serveur	: flaskapp-postgres	Configuration	: Expandible_B1ms_1_vCores_mémoire RAM Gio 2...
Emplacement	: France Central	Version de PostgreSQL	: 15.16
Statut	: Ready	Zone de disponibilité	: 2
Créé le	: 2026-03-12 10:58:16.0886985 UTC	Haute disponibilité	: Non activé
Étiquettes (modifier)	: Environment : dev ManagedBy : Terraform Project : flaskapp		

Commencer Propriétés Recommandations (1) Supervision Tutoriels

Démarez votre projet

Connectez-vous à votre base de données pour la première fois en quelques étapes simples.

4.4 Sécurité Réseau (NSG)

IP	SKU	Allocation
-----	-----	-----
4.233.118.182	Standard	Static

Port	Protocole	Usage
22	TCP	SSH - Administration de la VM
80	TCP	HTTP - Nginx reverse proxy
5000	TCP	Flask - Accès direct à l'application

4.5 Résultat terraform apply

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

```
flask_app_url           = "http://4.233.118.182:5000"
vm_public_ip           = "4.233.118.182"
vm_ssh_command         = "ssh azureuser@4.233.118.182"
storage_account_name   = "flaskappstorage61atxp"
storage_primary_endpoint = 
"https://flaskappstorage61atxp.blob.core.windows.net/"
postgres_fqdn          = "flaskapp-postgres.postgres.database.azure.com"
resource_group_name    = "rg-flask-app"
```

Après une relance:

```
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

flask_app_url = "http://4.233.118.182:5000"
postgres_connection_string = <sensitive>
postgres_fqdn = "flaskapp-postgres.postgres.database.azure.com"
resource_group_name = "rg-flask-app"
storage_account_name = "flaskappstorage61atxp"
storage_container_name = "static-files"
storage_primary_endpoint = "https://flaskappstorage61atxp.blob.core.windows.net/"
vm_public_ip = "4.233.118.182"
vm_ssh_command = "ssh azureuser@4.233.118.182"
```

5. Étape 3 — Backend Flask et Connexion au Stockage

5.1 Endpoints API

Méthode	Route	Description
GET	/health	Vérification état de l'application
POST	/files	Upload fichier → Blob Storage + métadonnées PostgreSQL
GET	/files	Liste tous les fichiers depuis PostgreSQL
GET	/files/<id>	Détail d'un fichier par ID
PUT	/files/<id>	Modifier la description d'un fichier
DELETE	/files/<id>	Supprimer le blob Azure + la ligne en BD

5.2 Modèle de données PostgreSQL

```
class FileMetadata(db.Model):  
    id = db.Column(db.String(36), primary_key=True) # UUID  
    filename = db.Column(db.String(255), nullable=False)  
    blob_url = db.Column(db.Text) # URL Azure Blob  
    size_bytes = db.Column(db.Integer)  
    uploaded_at = db.Column(db.DateTime, default=datetime.utcnow)  
    description = db.Column(db.Text)
```

5.3 Connexion Azure Blob Storage

```
def get_container():  
    conn =  
    f"DefaultEndpointsProtocol=https;AccountName={account};AccountKey={key};..."  
    return BlobServiceClient.from_connection_string(conn) \  
        .get_container_client(container_name)
```

5.4 Variables d'environnement sur la VM

```
# /opt/flaskapp/.env (chmod 600, owner: flaskuser)  
DATABASE_URL=postgresql://pgadmin:***@flaskapp-postgres....:5432/flaskdb  
AZURE_STORAGE_ACCOUNT=flaskappstorage61atxp  
AZURE_STORAGE_KEY=***  
AZURE_CONTAINER_NAME=static-files
```

6. Étape 4 — Automatisation du Déploiement

6.1 Variables dynamiques (variables.tf)

```
variable "admin_password" {  
  description = "Admin password for the virtual machine"  
  type        = string  
  sensitive   = true    # Masqué dans les outputs Terraform  
}  
  
variable "location" {  
  description = "Azure region"  
  type        = string  
  default     = "France Central"  
}
```

6.2 Provisioning automatique — Cloud-init

Le fichier scripts/cloud-init.yaml est injecté dans la VM via le champ custom_data (encodé en base64). Il automatise l'installation complète de l'environnement :

- Installation des packages système (Python 3, pip, Nginx, git)
- Création de l'utilisateur applicatif flaskuser (sans shell)
- Création de l'environnement virtuel Python (/opt/flaskapp/venv)
- Installation des dépendances Flask, SQLAlchemy, azure-storage-blob, gunicorn
- Configuration du service systemd pour démarrage automatique
- Configuration de Nginx comme reverse proxy

6.3 Service systemd

```
azureuser@flaskapp-vm:~$ cat /etc/systemd/system/flaskapp.service  
[Unit]  
Description=Flask Application  
After=network.target  
[Service]  
Type=simple  
User=flaskuser  
WorkingDirectory=/opt/flaskapp  
EnvironmentFile=/opt/flaskapp/.env  
ExecStart=/opt/flaskapp/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:5000 app:app  
Restart=always  
[Install]  
WantedBy=multi-user.target
```

6.4 Outputs Terraform

```
output "vm_public_ip" {
  value = azure_rm_public_ip.vm.ip_address
}

output "flask_app_url" {
  value = "http://${azure_rm_public_ip.vm.ip_address}:5000"
}

output "postgres_connection_string" {
  value      = "postgresql://..."
  sensitive = true
}
```

```
azureuser@flaskapp-vm:~$ systemctl status flaskapp
● flaskapp.service - Flask Application
   Loaded: loaded (/etc/systemd/system/flaskapp.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-03-12 12:02:39 UTC; 1h 5min ago
 Main PID: 5791 (gunicorn)
    Tasks: 4 (limit: 9522)
   Memory: 115.3M
      CPU: 1.604s
   CGroup: /system.slice/flaskapp.service
           └─5791 /opt/flaskapp/venv/bin/python3 /opt/flaskapp/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:5000 app:app
           └─5796 /opt/flaskapp/venv/bin/python3 /opt/flaskapp/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:5000 app:app
           └─5797 /opt/flaskapp/venv/bin/python3 /opt/flaskapp/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:5000 app:app

Mar 12 12:02:39 flaskapp-vm systemd[1]: Started Flask Application.
Mar 12 12:02:39 flaskapp-vm gunicorn[5791]: [2026-03-12 12:02:39 +0000] [5791] [INFO] Starting gunicorn 25.1.0
Mar 12 12:02:39 flaskapp-vm gunicorn[5791]: [2026-03-12 12:02:39 +0000] [5791] [INFO] Listening at: http://0.0.0.0:5000 (5791)
Mar 12 12:02:39 flaskapp-vm gunicorn[5791]: [2026-03-12 12:02:39 +0000] [5791] [INFO] Using worker: sync
Mar 12 12:02:39 flaskapp-vm gunicorn[5791]: [2026-03-12 12:02:39 +0000] [5791] [INFO] Control socket listening at /opt/flaskapp/gunicorn.ctl
Mar 12 12:02:39 flaskapp-vm gunicorn[5796]: [2026-03-12 12:02:39 +0000] [5796] [INFO] Booting worker with pid: 5796
Mar 12 12:02:39 flaskapp-vm gunicorn[5797]: [2026-03-12 12:02:39 +0000] [5797] [INFO] Booting worker with pid: 5797
```

```
C:\Users\haida\ccomputing\terraform-flask-azure\terraform>terraform output
flask_app_url = "http://4.233.118.182:5000"
postgres_connection_string = <sensitive>
postgres_fqdn = "flaskapp-postgres.postgres.database.azure.com"
resource_group_name = "rg-flask-app"
storage_account_name = "flaskappstorage61atxp"
storage_container_name = "static-files"
storage_primary_endpoint = "https://flaskappstorage61atxp.blob.core.windows.net/"
vm_public_ip = "4.233.118.182"
vm_ssh_command = "ssh azureuser@4.233.118.182"
```

7. Étape 5 — Tests et Validation

7.1 Health Check

```
$ curl http://4.233.118.182/health
```

```
{"status":"ok","time":"2026-03-12T12:00:30.820195"}
```

```
azureuser@flaskapp-vm:~$ curl http://4.233.118.182/health
{"status":"ok","time":"2026-03-12T13:09:15.729409"}
```

```
azureuser@flaskapp-vm:~$ curl http://4.233.118.182:5000/health
{"status":"ok","time":"2026-03-12T13:11:33.026657"}
```

7.2 CREATE — Upload d'un fichier

```
PS C:\Users\haida> curl.exe -X POST http://4.233.118.182/files -F "file=@test.txt" -F "description=Mon fichier de test"
curl: (26) Failed to open/read local data from file/application
PS C:\Users\haida> echo "contenu test" > test.txt
PS C:\Users\haida> curl.exe -X POST http://4.233.118.182/files -F "file=@test.txt" -F "description=Mon fichier de test"
{"blob_url":"https://flaskappstorage61atxp.blob.core.windows.net/static-files/b1d2c5ff-cad3-4fe0-8a37-7dffb478e761_test.txt","description":"Mon fichier de test","filename":"test.txt","id":"a14469ec-148a-44ec-9888-fddb9c085c8","size_bytes":30,"uploaded_at":"2026-03-12T13:38:48.906615"}
```

☐ Nom	Dernière modification	Niveau d'accès	Type d'objet bl...	Taille	État du bail
☐ 93b753dc-6d84-4d34-84b6-77974c7719f6_test.txt	12/03/2026 13:00:44	Chaud (déduit)	Objet blob de ...	30 B	Disponible ...
☐ b1d2c5ff-cad3-4fe0-8a37-7dffb478e761_test.txt	12/03/2026 14:38:48	Chaud (déduit)	Objet blob de ...	30 B	Disponible ...

☐ Nom	Dernière modification	Niveau d'accès	Type d'objet bl...	Taille	État du bail
☐ 93b753dc-6d84-4d34-84b6-77974c7719f6_test.txt	12/03/2026 13:00:44	Chaud (déduit)	Objet blob de ...	30 B	Disponible ...

7.3 READ — Liste des fichiers et lecture de fichier par ID

```
PS C:\Users\haida> Invoke-RestMethod http://4.233.118.182/files
```

```
blob_url      : https://flaskappstorage61atxp.blob.core.windows.net/static-files/b1d2c5ff-cad3-4fe0-8a37-7dffb478e761_test.txt
description   : Mon fichier de test
filename      : test.txt
id            : a14469ec-148a-44ec-9888-fddb9c085c8
size_bytes    : 30
uploaded_at   : 2026-03-12T13:38:48.906615
```

```
PS C:\Users\haida> Invoke-RestMethod http://4.233.118.182/files/a14469ec-148a-44ec-9888-fddb9c085c8
```

```
blob_url      : https://flaskappstorage61atxp.blob.core.windows.net/static-files/b1d2c5ff-cad3-4fe0-8a37-7dffb478e761_test.txt
description   : Mon fichier de test
filename      : test.txt
id            : a14469ec-148a-44ec-9888-fddb9c085c8
size_bytes    : 30
uploaded_at   : 2026-03-12T13:38:48.906615
```

7.4 UPDATE — Modification de la description

```
PS C:\Users\haida> Invoke-RestMethod -Method PUT http://4.233.118.182/files/a14469ec-148a-44ec-9888-fddb9c085c8 -ContentType "application/json" -Body '{"description":"Nouvelle description"}'
```

```
blob_url      : https://flaskappstorage61atxp.blob.core.windows.net/static-files/b1d2c5ff-cad3-4fe0-8a37-7dffb478e761_test.txt
description   : Nouvelle description
filename      : test.txt
id            : a14469ec-148a-44ec-9888-fddb9c085c8
size_bytes    : 30
uploaded_at   : 2026-03-12T13:38:48.906615
```

7.5 DELETE — Suppression

```
PS C:\Users\haida> Invoke-RestMethod -Method DELETE http://4.233.118.182/files/a14469ec-148a-44ec-9888-fddbe9c085c8
message
-----
deleted a14469ec-148a-44ec-9888-fddbe9c085c8
```

7.6 Destruction de l'infrastructure

```
Plan: 0 to add, 0 to change, 16 to destroy.
```

```
Changes to Outputs:
```

```
- flask_app_url           = "http://4.233.118.182:5000" -> null
- postgres_connection_string = (sensitive value) -> null
- postgres_fqdn           = "flaskapp-postgres.postgres.database.azure.com" -> null
- resource_group_name      = "rg-flask-app" -> null
- storage_account_name     = "flaskappstorage61atxp" -> null
- storage_container_name   = "static-files" -> null
- storage_primary_endpoint = "https://flaskappstorage61atxp.blob.core.windows.net/" ->
- vm_public_ip             = "4.233.118.182" -> null
- vm_ssh_command           = "ssh azureuser@4.233.118.182" -> null
```

```
Do you really want to destroy all resources?
```

```
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
```

```
00m20s elapsed]
```

```
azurerm_resource_group.main: Still destroying... [id=/subscription
```

```
00m30s elapsed]
```

```
azurerm_resource_group.main: Still destroying... [id=/subscription
```

```
00m40s elapsed]
```

```
azurerm_resource_group.main: Destruction complete after 48s
```

```
Destroy complete! Resources: 16 destroyed.
```

Mon tableau de bord

Tableau de bord privé

 Créer  Charger  Actualiser  Plein écran

Actualisation automatique : **Désactivé**

Toutes les ressources

Tous les abonnements

 Actualiser

 NetworkWatcher_francecent... Network Watcher

8. Problèmes Rencontrés et Solutions

Problème 1 — Service Principal non autorisé

Champ	Détail
Contexte	az ad sp create-for-rbac
Erreur	Insufficient privileges to complete the operation.
Cause	Le compte Azure for Students EFREI ne permet pas la création de Service Principals.
Solution	Utilisation de l'authentification directe az login dans le provider Terraform sans Service Principal.

Problème 2 — Régions non autorisées (Azure Policy)

Champ	Détail
Contexte	terraform apply
Erreur	RequestDisallowedByAzure: Resource was disallowed by Azure
Cause	La subscription EFREI applique une politique qui restreint les déploiements à 5 régions spécifiques : switzerlandnorth, polandcentral, swedencentral, francecentral, norwayeast.
Solution	Diagnostic via az policy assignment list pour identifier les régions autorisées. Changement vers France Central.

Problème 3 — Tailles de VM indisponibles (Capacity Restrictions)

Champ	Détail
Contexte	terraform apply
Erreur	SkuNotAvailable: Standard_B1s / B2s / DS1_v2 / D2s_v3 not available in FranceCentral
Cause	France Central avait des restrictions de capacité sur les tailles courantes au moment du déploiement.
Solution	Analyse des SKUs disponibles via l'API Azure. Utilisation de Standard_B2s_v2 avec zone=2.

Problème 4 — Conflit PostgreSQL Public/Private Network

Champ	Détail
Contexte	azurerm_postgresql_flexible_server
Erreur	ConflictingPublicNetworkAccessAndVirtualNetworkConfiguration
Cause	PostgreSQL Flexible Server avec delegated_subnet_id impose public_network_access_enabled = false.
Solution	Ajout explicite de public_network_access_enabled = false dans la ressource Terraform.

Problème 5 — Zone PostgreSQL non gérée par Terraform

Champ	Détail
Contexte	terraform apply (refresh)
Erreur	zone can only be changed when exchanged with standby_availability_zone
Cause	Azure assigne automatiquement une zone à PostgreSQL mais Terraform voulait la modifier.
Solution	Ajout d'un bloc lifecycle { ignore_changes = [zone] } dans la ressource.

Problème 6 — cloud-init : fichiers non créés (owner inexistant)

Champ	Détail
Contexte	cloud-init au boot VM
Erreur	ERROR: No such file or directory: '/opt/flaskapp/requirements.txt'
Cause	write_files spécifiait owner: flaskuser avant que cet utilisateur soit créé.
Solution	Contournement via az vm run-command invoke pour installer les dépendances directement sur la VM après démarrage.

Problème 7 — Table PostgreSQL inexistante au démarrage Gunicorn

Champ	Détail
Contexte	GET/POST /files
Erreur	ProgrammingError: relation 'file_metadata' does not exist
Cause	db.create_all() était dans if __name__ == '__main__', non exécuté par Gunicorn.
Solution	Déplacement de db.create_all() dans with app.app_context(): au niveau module.

9. Conclusion

9.1 Bilan technique

Composant	Statut	Détail
Infrastructure Terraform	Succès	16 ressources Azure déployées
VM Ubuntu avec IP publique	Succès	4.233.118.182 — Ubuntu 22.04
Azure Blob Storage	Succès	Container privé static-files
PostgreSQL managé	Succès	Flexible Server v15, accès privé
Application Flask	Succès	Gunicorn 2 workers + Nginx
Service systemd	Succès	Démarrage automatique au boot
API CRUD complète	Succès	5 endpoints testés et validés
Sécurité	Succès	Secrets hors Git, BD privée, Blob privé

9.2 Points clés appris

- Les comptes Azure for Students ont des restrictions importantes : régions limitées, pas de Service Principal, capacité réduite sur certaines tailles de VM.
- Le fichier terraform.tfstate contient des données sensibles (clés d'accès) et ne doit jamais être commité dans Git.
- cloud-init a des limitations d'ordre d'exécution — les fichiers avec owner d'un utilisateur inexistant ne sont pas créés.
- Gunicorn n'exécute pas le bloc `__main__` — l'initialisation de la DB doit être faite au niveau module.
- terraform import est essentiel pour réconcilier le state Terraform avec des ressources déjà existantes après un state drift.

9.3 Suppression de l'infrastructure

```
# Supprimer toutes les ressources Azure créées
cd terraform/
terraform destroy
# Taper "yes" pour confirmer — ~5 minutes
```